## LIST OF EXPERIMENTS

1. Analog (op amp based) simulation of linear differential equations.
2. Numerical Simulation of given nonlinear differential equations.
3. Real time simulation of differential equations.
4. Mathematical modeling and simulation of physical systems in at least two fields.
    a) Mechanical
    b) Electrical
    c) Chemical process
5. System Identification through process reaction curve.
6. Stability analysis using Pole zero maps and Routh Hurwitz Criterion in simulation platform.
7. Root Locus based analysis in simulation platform.
8. Determination of transfer function of a physical system using frequency response and Bodes asymptotes.
9. Design of Lag, lead compensators and evaluation of closed loop performance.
10. Design of PID controllers and evaluation of closed loop performance.
11. Discretization of continuous system and effect of sampling.
12. Test of controllability and observability in continuous and discrete domain in simulation platform.
13. State feedback and state observer design and evaluation of closed loop performance.

## MINI PROJECTS

**Mini Project 1:** Simulation of complete closed loop control systems including sensor and actuator dynamics.

**Mini Project 2:** Demonstration of a closed loop system in hardware.

**SIMULATION DIAGRAM:**

**Ex. No.:** **MATHEMATICAL MODELLING AND SIMULATION OF MECHANICAL TRANSLATIONAL SYSTEM**

**Date :**

**AIM:**

To construct the simulation block diagram of mechanical translational system and analyze its dynamic behavior and response to external force.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation.

**DESIGN:**

Consider the second order mechanical translation system is shown in Fig. 1.1.

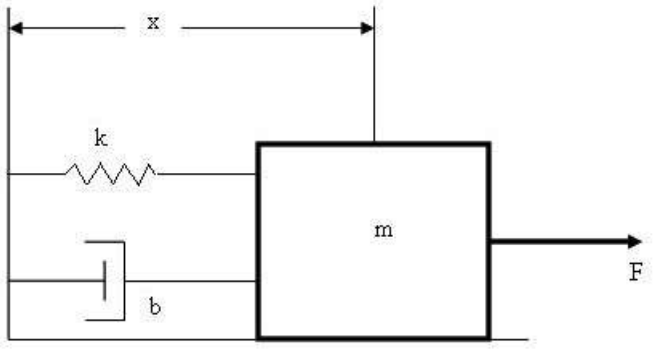



Fig.1.1 The mechanical system

In the given system, applied force $F(t)$ is the input and displacement $x$ is the output. The system has one node i.e., mass $M$. The differential equation governing the system is given by the force balance equation.

$$F(t) = M\frac{d^2x}{dt^2} + B\frac{dx}{dt} + K\,x \tag{1}$$

$$\frac{d^2x}{dt^2} = \frac{F(t)}{M} - \frac{B}{M}\frac{dx}{dt} - \frac{K}{M}x \tag{2}$$

Taking Laplace transform of above Eq. (1), and to find the transfer function of translational mechanical system,

$$F(s) = M\,s^2\,x(s) + B\,s\,x(s) + Kx(s) \tag{3}$$

**SIMULATION RESULT:**

The transfer function of the mechanical translational system is given by,

$$\frac{x(s)}{F(s)} = \frac{1}{M s^2 + B s + K} \quad (4)$$

Let us consider the values are,

$M = 0.33Kg$

$K = 1.65$

$B = 0.66$

By substituting the above values, the transfer function is obtained as,

$$\frac{x(s)}{F(s)} = \frac{1}{0.33 s^2 + 0.66 s + 1.65}$$

**PROCEDURE:**

1. Click on MATLAB icon and Select SIMULINK icon to open Simulink Library Browser.

2. From FILE menu click on NEW button and select MODEL to open untitled window.

3. Create a Simulink Model by using the below equation,

$$\frac{d^2x}{dt^2} = \frac{F(t)}{M} - \frac{B}{M}\frac{dx}{dt} - \frac{K}{M}x$$

4. Click the START SIMULATION icon to run the Simulation Model.

5. Obtain the response from SCOPE.

**INFERENCE:**

**RESULT:**

**SIMULATION DIAGRAM:**

**Ex. No.:** **MATHEMATICAL MODELLING AND SIMULATION OF CHEMICAL PROCESS**

**Date:**

**AIM:**

To construct the simulation block diagram of mechanical translational system and analyze its dynamic behavior and response to external input flow.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation

**DESIGN:**

For a pure capacity single level process as shown in Fig. 1. 2.

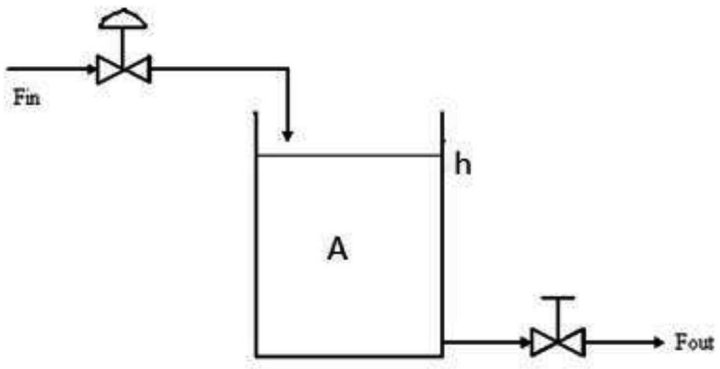



**Fig.1.2 Level control of single tank process**

The Mass balance equation of above tank process is,

$$F_{in} - F_{out} = A\frac{dh}{dt} \quad (1)$$

Where, A is the area of the tank and h is the height of the tank to control the process. For this tank process $F_{Out} = \frac{h}{R}$ sub in above equation,

$$F_{in} - \frac{h}{R} = A\frac{dh}{dt} \quad (2)$$

$$h = \frac{1}{A}\int\left(F_{in} - \frac{h}{R}\right)dt$$

Laplace transform of the above equation is taken to find the transfer function and using that transfer function, the tank level process is simulated. Step input is given to the process and the variation of height in tank is checked using the scope.

Let us consider the values are,

Area of the tank $A = 5\ m^2$

Flow resistance $R = 0.2$

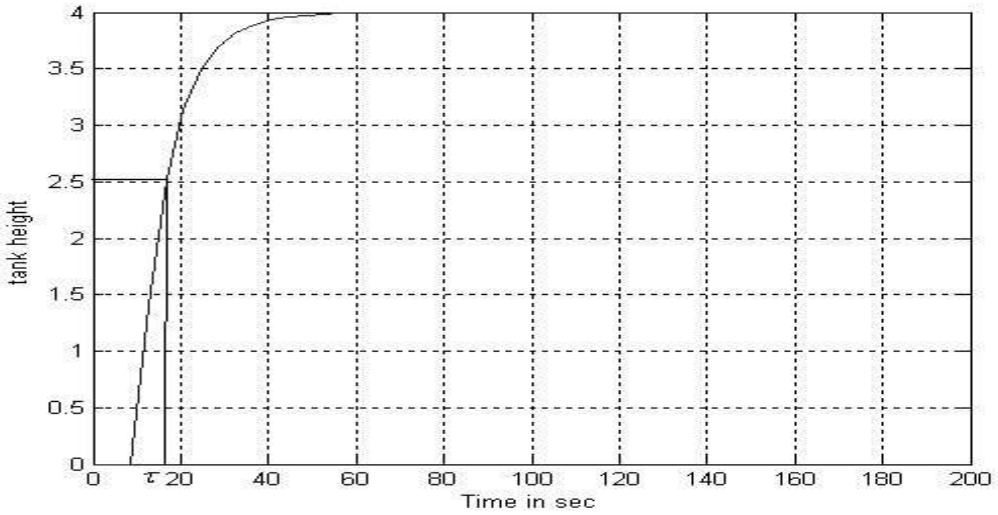



**Fig. 1.3 Servo response of a tank level system**

**SIMULATION RESULT:**

**PROCEDURE:**

1. Click on MATLAB icon and Select SIMULINK icon to open Simulink Library Browser.

2. From FILE menu click on NEW button and select MODEL to open untitled window.

3. Create a Simulink Model by using the below equation,

$$h = \frac{1}{A}\int\left(F_{in} - \frac{h}{R}\right)dt$$

$$h = \frac{1}{A}\int\left(F_{in} - \frac{h}{R}\right)dt$$

4. Click the START SIMULATION icon to run the Simulation Model.

5. Obtain the response from SCOPE.

**INFERENCE:**

**RESULT:**

**SIMULATION DIAGRAM:**

**EX.NO: REALTIME SIMULATION OF DIFFERENTIAL EQUATION**

**DATE:**

**AIM:**

To simulate the differential equation of a series RLC circuit using MATLAB.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation

**DESIGN:**

In a series RLC circuit, the resistor (R), inductor (L), and capacitor (C) are connected in a single loop. The instantaneous current flowing through the circuit is the same for all components. Because the inductive reactance and capacitive reactance are frequency-dependent, the circuit's response varies with different input frequencies. This variation affects how the voltage drops across each element are phase-shifted relative to each other.

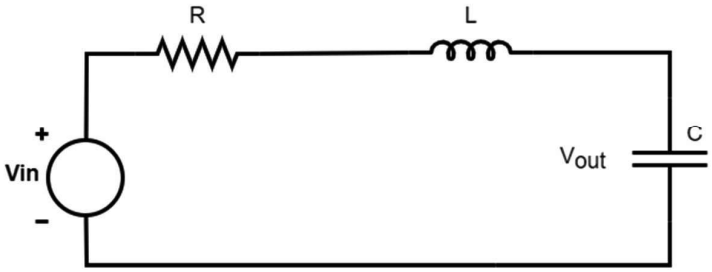



**Fig.1 RLC Series Circuit**

In a series RLC circuit, the components Resistor R, inductor L, and capacitor C are connected in series. The instantaneous voltage and current relationships are governed by the differential Eq. (1):

$$V_{in} = iR + L\frac{di}{dt} + \frac{1}{C}\int_0^t idt \quad (1)$$

$$V_{out} = \frac{1}{C}\int_0^t idt \quad (2)$$

The transfer function of the RLC series circuit is given in Eq. (3)

$$\frac{V_{out}(s)}{V_{in}(s)} = \frac{1/LC}{S^2 + \frac{R}{L}S + 1/LC} \quad (3)$$

**SIMULATION RESPONSE:**

Simulates the series RLC circuit and derive the transfer function of a series RLC circuit with the given values, then obtain the responses.

In a series RLC circuit with the given values:

- Resistance (R) = 2.5 ohms
- Inductance (L) = 9 H
- Capacitance (C) = 50 µF

**PROCEDURE:**

1. Click on MATLAB icon and Select SIMULINK icon to open Simulink Library Browser.
2. From FILE menu click on NEW button and select MODEL to open untitled window.
3. Create a Simulink Model by using Powersim tool.
4. Add a Transfer Function Block and set its numerator and denominator based on the RLC circuit parameters.
5. Click the START SIMULATION icon to run the Simulation Model.
6. Obtain the response from SCOPE.

**INFERENCE:**

**RESULT:**

## SIMULATION

**M-File:**

```
f = input('Enter function: ');
t0 = input('Enter initial value of independent variable: ');
y0 = input('Enter initial value of dependent variable: ');
h = input('Enter step size: ');
tn = input('Enter point at which you want to evaluate: ');
n=(tn-t0)/h;
t(1)=t0;
y(1)=y0;
for i=1:n
    y(i+1)=y(i)+h*f(t(i),y(i));
    t(i+1)=t0+i*h;
    fprintf('y(%0.2f)= %0.4f\n',t(i+1),y(i+1))
end
```

## SIMULATION DIAGRAM

**EX.NO: NUMERICAL SIMULATION OF NONLINEAR DIFFERENTIAL EQUATION**

**DATE:**

**AIM:**

To apply Euler's method to numerically solve a given non-linear differential equation and visualize the results using MATLAB.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation

**DESIGN:**

Most of the differential equations, representing physical systems, are non-linear and, hence, not analytically solvable. Although there are numerous techniques for finding the analytic solution of first order differential equations, we are unable to easily obtain closed analytic solutions for many equations. Then numerical methods become necessity. This experiment discusses the underlying concepts which are common to most of numerical methods for solving ODEs.

Let consider a general first order ODE

$\frac{dy}{dt} = f(x, y)$, with the initial condition $y\ (x=x_0) = y_0$ find y(x) for $x_0 < x < L$

Euler's method is one of the simplest and most fundamental numerical techniques used to approximate solutions of ordinary differential equations (ODEs). Also, Euler's method provides a simple yet effective approach for solving non-linear ODEs. The experiment demonstrates that accuracy improves with smaller step sizes with increased computational effort. The basic idea of Euler's method is to use the slope of the differential equation to project the solution forward in small steps. Euler's method approximates the solution using the formula:

$$y_{n+1} = y_n + h \cdot f(t_n, y_n) \quad (1)$$

Where, $f(t_n, y_n)$ is the ODE function. The function or system with variables, in reality, the variables x and y are continuous. Hence, to calculate the values of $y$ at infinite values (points) of $x$, which is not possible by computers. As a result, in all the numerical methods, we only calculate the values of y at the finite values of x. It involves the following steps:

1. **Discretization:**

- Divide the time interval $[t_0, t_f]$ into N small steps of size $h$, where $h$ is the step size and $t_{n+1} = t_n + h$.
- The solution at each step is approximated using the slope of $f(t,y)$ at the current point.

**PROBLEM:**

The non-linear dynamic system is represented as,

$$\frac{dy}{dt} + 20y = 7e^{-0.5t} \quad ; y(0) = 5 \tag{1}$$

Compute y(0.2) using Eulers method.

To use Euler's method to compute y(0.2)y(0.2)y(0.2) for a non-linear dynamic system and need the following information:

1. **The differential equation** that describes the system.
2. **The initial condition** y(0).
3. **The step size** h for Euler's method.

The given ODE can be written as,

$$\frac{dy}{dt} = -20y + 7e^{-0.5t} \tag{2}$$

The initial condition is typically $y_0$=5, $t_0$=0.

The step size $h$ is generally chosen based on the desired accuracy. Here, the time span of interest is from t=0 to t=0.2, and we need to perform iterations to estimate y (0.2).

**Eulers Method:**

Euler's method approximates the solution using the below equations:

$$y_{n+1} = y_n + hf(t_n, y_n) \tag{4}$$

$$t_n = t_0 + nh \tag{5}$$

**First Iteration, n=0**

$$y_1 = y_0 + hf(t_0, y_0)$$
$$y_1 = 5 + 0.1f(0,5)$$
$$y_1 = 5 + 0.1*(-93)$$
$$y_1 = -4.3$$

**Second Iteration, n=1**

$$t_1 = t_0 + 1h$$
$$t_1 = 0 + 1*(0.1)$$
$$t_1 = 0.1$$
$$y_2 = y_0 + hf(t_1, y_1)$$
$$y_2 = 5 + 0.1f(0.1, -4.3)$$
$$y_1 = 5 + 0.1*(-92.65)$$
$$y_2 = 4.96$$

2. **Update Formula:**
    - Compute the next value $y$ using the formula: $y_{n+1}=y_n + h \cdot f(t_n,y_n)$
    - Here, $y_n$ is the current value, $t_n$ is the current time, and $f(t_n,y_n)$ is the derivative at the current point.
3. **Iteration**
    - Repeat the update formula for each step from $t_0$ to $t_f$ to generate the numerical solution.

To solve the non-linear ODE using Euler's method and it involves the three stages:

1. **Define Parameters:**
    - Initial time $t_0$
    - Initial value $y_0$
    - Step size $h$
    - Final time $t_f$
2. **Apply Euler's Method:**

    At each step, compute $y_{n+1}= y_n + h\cdot f(t_n, y_n)$ and $t_{n+1} = t_n + h$
3. **Iterate Through Steps:**
    - Update the values from $t_0$ to $t_f$ and plot or analyze the results.

**PROBLEM:**

A tank with a constant cross-sectional area of $A=1\ m^2$. The tank starts with an initial fluid height of $h(0)=1$ meter. Fluid flows into the tank at a rate of $Q_{in}(t)=1$ cubic meters per second, and it flows out at a rate of $Q_{out}(t)= b * \sqrt{h}$ cubic meters per second and the flow coefficient $b$ is 2. Using the Euler's method to approximate the fluid level in a single tank.

**PROCEDURE:**

1. Identify the ODE to be solved, for example: $dy/dt=f(t,y)$ and define the initial condition: $y(t_0)=y_0$ .
2. Determine the following parameters:

   Initial Time ($t_0$): The starting point of the time interval.

   Final Time ($t_f$): The end point of the time interval.

   Step Size ($h$): The increment for each iteration, affecting the accuracy of the solution
3. Start MATLAB and create a new script or live script for your calculations.
4. Use a for loop to iteratively update the solution using Euler's formula:
5. Run and plot to visualize the numerical the solution.
6. Create a simulation diagram and run the simulation model.
7. Obtain the responses from SCOPE and compare the results.

**INFERENCE:**

**RESULT:**

**PROBLEM:**

Consider the second order differential equation.

$$10\frac{d^2v}{dt^2}+200\frac{dv}{dt}+1000\,v(t)=250V$$

Simulate the given deferential equation using op-amp.

**EX.NO: 4 ANALOG (OP AMP BASED) SIMULATION OF LINEAR DIFFERENTIAL EQUATIONS**

**DATE:**

**AIM:**

To implement analog circuit using operational amplifiers (op-amps) to simulate linear differential equations and obtain the response of a circuit model and simulation model.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation

**DESIGN:**

The linear differential equations can be simulated using analog circuits by utilizing op-amps for performing integration and summation. This approach involves designing circuits that implement these operations, then combining them to model the behavior described by the differential equation. The general form of a second-order linear differential equation is:

$$\frac{d^2y}{dt^2} + a_1 \frac{dy}{dt} + a_0\, y(t) = b_0\, u(t) \qquad (1)$$

The op amp circuit can solve mathematical equations fast, including calculus problems such as differential equations. To solve a differential equation by finding *y(t)*, use various op-amp configurations to find the output voltage. The inverting op-amp is shown in Figure 1.

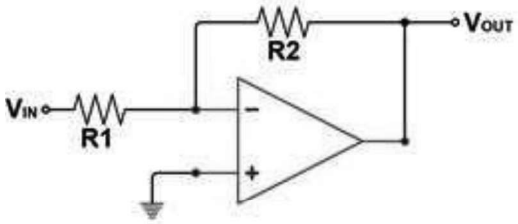



Fig. 1 An inverting op-amp

The output voltage equation of an inverting op-amp is,

$$V_{out} = \frac{-R2}{R1} V_{in} \qquad (2)$$

Op-amps can be configured as differentiators and integrators to model these equations. A differentiator circuit computes the derivative of a signal, while an integrator circuit computes the integral. To simulate the differential equation using op-amps, this circuit can compute the derivatives and integrals of signals, and a summing circuit to combine these according to the coefficients.

**SIMULATION DIAGARM:**

An op-amp based differentiator typically consists of a capacitor C in series with the input signal and a resistor R in the feedback path. The output of this circuit is proportional to the derivative of the input signal:

$$V_{out} = -RC\frac{dV_{in}}{dt} \tag{3}$$

An op-amp based integrator has a resistor R at the input and a capacitor C in the feedback loop. The output of this circuit is proportional to the integral of the input signal:

$$V_{out} = \frac{-1}{RC}\int V_{in}\, dt \tag{4}$$

Also, op-amp configured as a summing amplifier can combine multiple input signals, each weighted by different coefficients. The inputs are connected through resistors to the inverting input of the op-amp, with a feedback resistor.

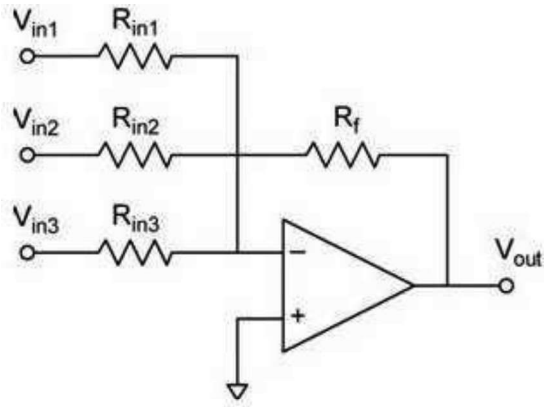



Fig. 2. Summing amplifier

The output is a weighted sum of the inputs:

$$V_{out} = -\left(\frac{R_f}{R_1}V_1 + \frac{R_f}{R_2}V_2 + \text{++++}\right) \tag{5}$$

Where, $R_f$ is the feedback resistor and $R_1$, $R_2$,…are the input resistors. Choose resistor values to achieve the correct scaling of inputs based on the coefficients $a_0$, $a_1$and $b_0$.

**PROCEDURE:**

1. Click on MATLAB icon and Select SIMULINK icon to open Simulink Library Browser.
2. From FILE menu click on NEW button and select MODEL to open untitled window.
3. Create a Simulink Model by using Electrical System tool.
4. Add a Transfer Function Block and set its numerator and denominator based on differential equation.
5. Click the START SIMULATION icon to run the Simulation Model.
6. Obtain the response from SCOPE.

**INFERENCE:**

**RESULT:**

**PROBLEM:**

Consider a unity feedback system with an open-loop transfer function:

$$G(s) = \frac{K}{s(s+4)(s+6)}$$

Obtain the root locus plot for G(s) and analyses the effect of varying system gain (K) on the location of the poles.

**EX.NO: 5 ROOT LOCUS BASED ANALYSIS IN SIMULATION PLATFORM**

**DATE:**

**AIM:**

To obtain the root locus plot for system transfer function and analyses the effect of varying system gain (K) on the location of the poles.

**SIMULATION Needs:**

1. Software : MATLAB Package
2. Tool : Simulation

**ROOT LOCUS:**

The Root Locus is a graphical technique used in control system design and analysis to examine how the poles of a closed-loop transfer function change as a system parameter, typically the gain (K), is varied. It provides a powerful way to understand the relationship between system stability, performance, and controller design.

The locations of poles and zeros of a transfer function play a crucial role in system behavior. Poles determine the system's stability and transient response, while zeros shape the system's frequency response.

The closed-loop transfer function is given by:

$$T(s) = \frac{K\,G(s)}{1+K\,G(s)\,H(s)} \qquad (1)$$

Where,

G(s) is the open-loop transfer function of the plant.

H(s) is the feedback transfer function (typically $H(s)=1$ for unity feedback).

K is the system gain.

The Root Locus examines the movement of the poles of T(s) as K is varied. The root locus plot illustrates how the roots of the characteristic equation vary as the feedback gain K changes:

$$1+KG(s)H(s)=0 \qquad (2)$$

The poles of the closed-loop system are solutions to this equation, and the root locus helps us visualize their movement.

**SIMULATION RESPONSE:**

**PROCEDURE:**

1. Click on MATLAB icon and Select COMMAND window.
2. Enter the following commands in command window.

```
>> a=tf([num],[den])
>> bode (a)
>> rlocus(a)
```

3. Run the above commands and obtain the Bode and Root locus plots.

**INFERENCE:**

**RESULT:**

**SIMULATION:**

**EX.NO: 6 SYSTEM IDENTIFICATION THROUGH PROCESS REACTION CURVE**

**DATE:**

**AIM:**

Obtain the dynamic response of a single tank process by applying a unit step input and determine the system parameters of a First Order Plus Dead Time (FOPDT) model.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation

**PROCESS REACTION CURVE METHOD:**

The Process Reaction Curve method involves applying a step input (change in input) to the system and measuring the output response. From this output, the following characteristics are extracted:

- Dead Time: The time delay before the system starts responding.
- Time Constant: The time it takes for the system to respond to about 63% of its final value.
- Gain: The ratio of the change in output to the change in input after the system has stabilized.

For a First Order Plus Dead Time (FOPDT) model, the transfer function is:

$$G(s) = \frac{K\, e^{-Ls}}{\tau s + 1} \qquad (1)$$

Where,

$\tau$ - Time taken to reach 63.2% of the final value

$K$- Ratio of output change to input change (Final stead state value of open loop response)

L is the dead time (delay) to reach the 10% of the final value

**DESIGN**

For a pure capacity single level process as shown in Fig. 1 delay time is introduced in inlet flow of tank process.

**MODEL GRAPH:**

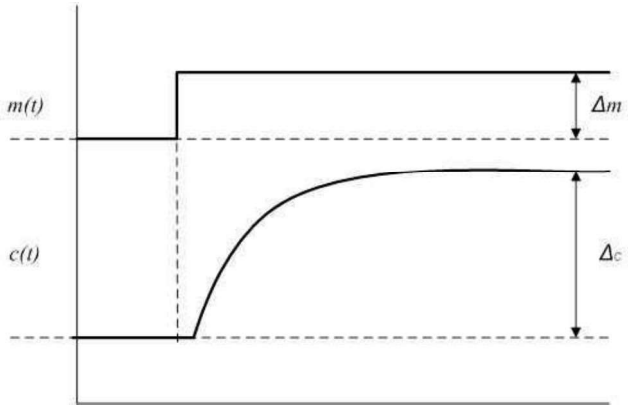



Fig. 2 Process reaction curve

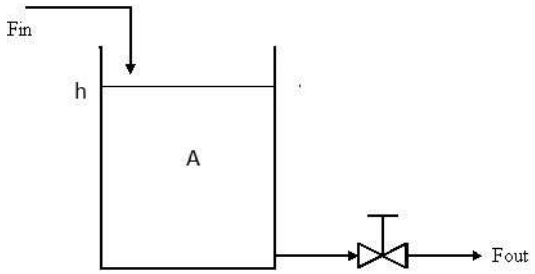



Fig.1 Single tank process

Mass balance equation of above tank process is,

$$F_{in} - b\sqrt{h} = A\frac{dh}{dt} \tag{2}$$

Where A is the area of the tank and h is the height of the tank to control the process

$$h(t) = \frac{1}{A}\int F_{in} - b\sqrt{h} \tag{3}$$

Laplace transform of the above equation is taken to find the transfer function and using that transfer function, the tank level process is simulated.

Consider the process parameters,

Area of the tank A= 0.5 $m^2$

Flow coefficient b=0.5

**PROCEDURE:**

1. Click on MATLAB icon and Select SIMULINK icon to open Simulink Library Browser.
2. From FILE menu click on NEW button and select MODEL to open untitled window.
3. Create a Simulink Model by using the below equation (2),
4. Click the START SIMULATION icon to run the Simulation Model.
5. Obtain the response from SCOPE.

**SIMULATION RESPONSE:**

**INFERENCE:**

**RSULT:**

**SIMULATION:**

**EX.NO: 7 DESIGN OF PID CONTROLLERS AND EVALUATION OF CLOSED LOOP PERFORMANCE**

**DATE:**

**AIM**

To design and implement Proportional-Integral (PI) and Proportional-Integral-Derivative (PID) controllers for level control in a single-tank process using the parameters identified through system identification.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation

**DESIGN:**

For a pure capacity single level process as shown in Fig. 1 delay time is introduced in inlet flow of tank process.

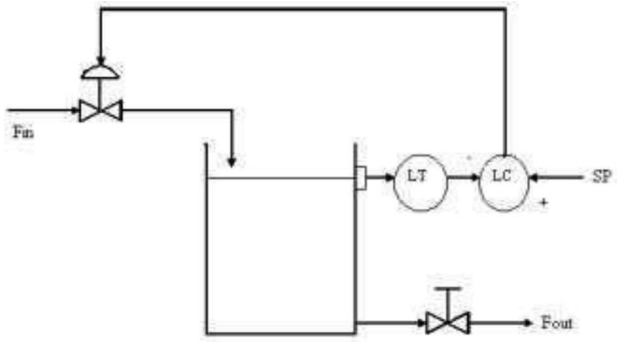



Fig.1 Level control of single tank process

Mass balance equation of above tank process is,

$$F_{in} - b\sqrt{h} = A\frac{dh}{dt} \quad (1)$$

Where, A is the area of the tank and h is the height of the tank to control the process. Consider the process parameters,

Area of the tank A= 0.5 $m^2$

Flow coefficient b=0.5

**SIMULATION RESPONSE:**

Transfer function of single tank level process is given by,

$$G(s) = \frac{K\, e^{-Ls}}{\tau s + 1} \qquad (2)$$

Calculate settings for Controller Gain (*Kp*), Integral Time (*Ti*), and Derivative Time (*Td*), using the Ziegler-Nichols tuning rules below.

For P control: $K_P = \frac{\tau}{L * K}$

For PI control: $K_P = \frac{0.9*\tau}{L * K}$ ; $K_i = \frac{Kp}{Ti}$ & $Ti = 3.33 * L$

For PID control: $K_P = \frac{1.2*\tau}{L * K}$ ; $K_i = \frac{Kp}{Ti}$ & $Ti = 2 * L$ ; $Kd = KP*Td$ & $Td = 0.5 * L$

**PROCEDURE:**

1. Click on MATLAB icon and Select SIMULINK icon to open Simulink Library Browser.
2. From FILE menu click on NEW button and select MODEL to open untitled window.
3. Create a Simulink Model with PI, PID controller and Feedback structure.
4. Click the START SIMULATION icon to run the Simulation Model.
5. Obtain the response from SCOPE.

**INFERENCE:**

**RSULT:**

**Problem:**

For the system represented by the following characteristic equations are given below. Using Routh Hurwitz stability criterion, Examine the necessary condition for stability and Determine the number of roots lying on right and left half of s- plane.

(i) $s^5+s^4+2s^3+2s^2+3s+5 = 0.$

(ii) $s^7 + 9s^6 + 24s^5+ 24s^4+24s^3+24s^2+23s+15 = 0.$

**EX.NO: STABILITY ANALYSIS USING POLE ZERO MAPS AND ROUTH HURWITZ CRITERION IN SIMULATION PLATFORM**

**DATE:**

**AIM:**

To verify the stability of a system using pole zero maps and Routh-Hurwitz Criterion using MATLAB.

**APPARATUS REQUIRED:**

1. Software : MATLAB Package
2. Tool : Simulation.

**THEORY:**

The theory of network synthesis states that any pole of the system lies on the right-hand side of the origin of the s plane, it makes the system unstable. On the basis of this condition A. Hurwitz and E.J. Routh started investigating the necessary and sufficient conditions of stability of a system. A first criterion is given by A. Hurwitz and this criterion is also known as Hurwitz Criterion for stability or Routh Hurwitz Stability Criterion. With the help of characteristic equation, we will make a number of Hurwitz determinants in order to find out the stability of the system. We define characteristic equation of the system as:

$a_0s^n + a_1s^{n-1} + a_2s^{n-2} + \ldots\ldots\ldots\ldots + a_0 = 0$

where, *n* determinants for nth order characteristic equation.

**PROCEDURE:**

1. Enter the program in the editor window.
2. Execute the program.
3. Enter the coefficients of characteristic equation in the command window.
4. Finally save the program and run it.
5. The Routh matrix is obtained in the command window.

**MATLAB CODE:**

```
clear
clc
%% firstly it is required to get first two row of routh matrix
e=input('enter the coefficients of characteristic equation: '); disp('--
- - - -
```

**SIMULATION RESULTS:**

**INFERENCE:**

**RESULT:**

**PROBLEM:**

The open loop transfer function of a unity feedback system $G(s) = K /s(s^2+2s+3)$. Obtain the Bode Plot. Find (i) Gain Margin (ii) Phase Margin (iii)Gain cross over frequency (iv) Phase cross over frequency (v) Resonant Peak (vi) Resonant Frequency (vii)Bandwidth and Check the same results using MATLAB Software. (Assume K=1)

**EX.NO: FREQUENCY RESPONSE ANALYSIS USING BODE PLOT**

**DATE:**

**AIM:**

To analyze the stability of the given linear system using Bode plot.

**APPARATUS REQUIRED:**

1. Software : MATLAB Package
2. Tool : Simulation.

**THEORY:**

The bode plot is a frequency response plot of the transfer function of a system. A bode plot consists of two graphs. One is plot of the magnitude of a sinusoidal transfer function versus log ω. The other is plot of the phase angle of a sinusoidal transfer function versus logω . The main advantage of the bode plot is that multiplication of magnitude can be converted into addition.

**PROCEDURE:**

1. Enter the program in the editor window.
2. Execute the program.
3. Finally save the program and run it.
4. The Routh matrix is obtained in the command window.

**MATLAB CODE:**

```
num=[1 ];
den=[1 2 3 0];
w=logspace(-1,3,100);
figure(1);
bode(num,den,w);
title('Bode Plot for the given transfer function G(s)=1/s(s^2+2s+3)')
grid;
[Gm Pm Wcg Wcp] =margin(num,den);
Gain_Margin_dB=20*log10(Gm)
Phase_Margin=Pm
Gaincrossover_Frequency=Wcp
Phasecrossover_Frequency=Wcg
[M P w]=bode(num,den);
[Mp i]=max(M);
Resonant_PeakdB=20*log10(Mp)
Wp=w(i);
Resonant_Frequency=Wp
for i=1:1:length(M);
if M(i)<=1/(sqrt(2));
Bandwidth=w(i)
break;
end;
end;
```

**SIMULATION RESULTS:**

**INFERENCE:**

**RESULT:**

**Problem (Design of Lag Compensator):**

Consider a unity feedback system has an open loop transfer function, $G(s) = \frac{K}{s(s+4)(s+80)}$.

Determine the system gain K for the velocity error constant is less than or equal to 30 $sec^{-1}$. Find the transfer function lag compensator for the system *G(s)*. It is desired to have the phase margin is 33°.

**EX.NO: DESIGN OF LAG, LEAD COMPENSATORS AND EVALUATION OF CLOSED LOOP PERFORMANCE**

**DATE:**

**AIM:**

To design a Lag and Lead Compensator using frequency response plot and evaluate the closed performance of a compensated system.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation

**DESIGN:**

**(i) LAG COMPENSATOR:**

The Lag compensator is designed by using the above transfer function G(s). The following steps are used to obtain the lag compensator transfer function.

1. Obtain the bode plot for the process transfer function G(s). From the plot determine the phase margin and gain margin.
2. Choose the suitable desired phase margin value and determine the lead compensator transfer function $G_c(s)$.
3. Determine the new phase margin ($\gamma_n$).

$$\gamma_n = \gamma_d + 5^o \quad (1)$$

Where, $\gamma_d$ is the desired phase margin

4. Find new gain cross over frequency, $\omega_{gcn}$, which is the frequency corresponding to $\gamma_n$ of previous step for that find $\phi_{gcn}$.

$$\phi_{gcn} = \gamma_n - 180 .$$

5. From the bode plot determine, $\omega_{gcn}$ corresponding to $\phi_{gcn}$.
6. Determine gain $A_{gcn}$ corresponding to $\omega_{gcn}$ from bode plot.
7. Find the compensator parameter $\beta$.

$$\beta = 10^{\frac{A_{gcn}}{20}} \quad (2)$$

8. Determine the compensator parameter T,

$$T = \frac{10}{\omega_{gcn}} \quad (3)$$

**Problem (Design of Lead Compensator):**

Consider a unity feedback system has an open loop transfer function, $G(s) = \frac{K}{s(s+1)(s+5)}$. Find the system gain K for the velocity error constant is $\leq 50$. Obtain the transfer function lead compensator for the system *G(s)*. It is desired to have the phase margin $\geq 20°$.

9. The transfer function of Lag compensator $G_c(s)$ is,

$$G_c(s) = \beta \frac{Ts+1}{\beta Ts+1} \tag{4}$$

10. The closed loop structure of compensated system is shown in Fig.1.

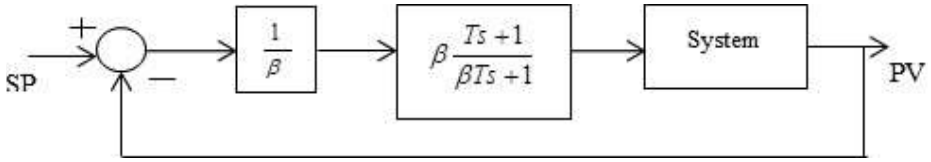



Fig. 1. Lag compensated system

**(ii) Lead Compensator:**

The lead compensator is designed by using the above transfer function G(s). The following steps are used to obtain the lead compensator transfer function.

1. Obtain the bode plot for the process transfer function G(s). From the plot determine the phase margin and gain margin.
2. Choose the suitable desired phase margin value and determine the lead compensator transfer function $G_c(s)$.
3. Determine the maximum phase lead angle, $\phi_m$.

$$\phi_m = \gamma_d - \gamma + 5^o \tag{5}$$

Where, $\gamma_d$ is the desired phase margin and $\gamma$ is the actual phase margin

4. Find the compensator parameter $\alpha$.

$$\alpha = \frac{1-\sin\phi_m}{1+\sin\phi_m} \tag{6}$$

5. Determine the compensator parameter T,

$$T = \frac{1}{\omega_m \sqrt{\alpha}} \tag{7}$$

Where, $\omega_m$ is maximum phase lead angular frequency.

$$\omega_m = -20\log_{10}\left(\frac{1}{\sqrt{\alpha}}\right) \text{ in db.} \tag{8}$$

The above equation is used to obtain value of $\omega_m$ in terms of decibel and the corresponding value of $\omega_m$ in rad/sec is obtained from the bode plot.

**SIMULATION RESULTS:**

6. The transfer function of Lead compensator $G_c(s)$ is,

$$G_c(s) = \alpha \frac{Ts+1}{\alpha Ts+1} \tag{9}$$

7. The closed loop structure of compensated system is shown in Fig.2.

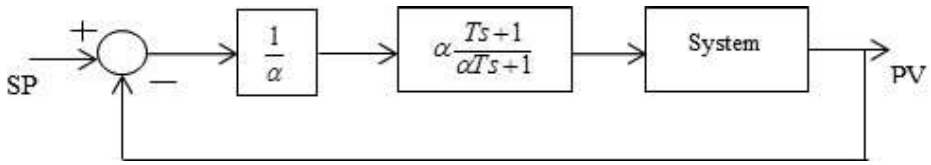



Fig. 2. Lead compensated system

**PROCEDURE:**

1. Click on MATLAB icon and Select SIMULINK icon to open Simulink Library Browser.
2. From FILE menu click on NEW button and select MODEL to open untitled window.
3. Create a Simulink Model by using the transfer functions of Lag (Fig.1) and Lead (Fig. 2) compensators.
4. Click the START SIMULATION icon to run the Simulation Model.
5. Obtain the response from SCOPE.

**INFERENCE:**

**RESULT:**

**Problem:**

Discretize the following continuous-time transfer function, $G(s) = \frac{10}{s^2 + 3s + 10} e^{-10s}$. Analyze the impact of different sampling periods on the system behavior.

**Matlab Commands:**

>>

**EX.NO: DISCRETIZATION OF CONTINUOUS SYSTEM AND EFFECT OF SAMPLING**

**DATE:**

**AIM:**

To discretize a given continuous-time control system and analyze the impact of different sampling periods on the system behavior.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation

**THEORY**

**1. Continuous-Time vs. Discrete-Time Systems**

- **Continuous-Time Systems**: Systems where signals change continuously over time, typically represented by differential equations or continuous-time transfer functions. In control theory, these systems are often modelled in the Laplace domain (using s-domain representations).
- **Discrete-Time Systems**: Systems that operate on signals at specific time intervals (samples) and are described by difference equations or discrete-time transfer functions. These systems are analysed in the z-domain.

In digital control, real-world continuous signals are sampled at regular intervals for processing by digital controllers, resulting in a discrete-time approximation of the original continuous system.

**2. Sampling and Nyquist Criterion**

- **Sampling**: The process of converting a continuous signal into a sequence of discrete samples by capturing values at regular intervals. The interval between samples is the sampling period Ts, while the frequency at which samples are taken is the sampling frequency fs=1/Ts.
- **Nyquist Criterion**: To avoid distortion (aliasing), the sampling frequency must be at least twice the maximum frequency present in the original continuous signal. This criterion ensures that the sampled data sufficiently represent the original continuous signal.

**3. Effect of Sampling Rate on System Behaviour**

- **High Sampling Rate**: When the sampling rate is high (small Ts), the discrete system can closely approximate the continuous system's behavior.
- **Low Sampling Rate**: With a low sampling rate (large Ts), the discrete approximation becomes less accurate.

**Simulation Diagram:**

**4. Step Response Analysis**

- A common method to evaluate the accuracy of discretized systems is by comparing the step response of the continuous system to that of the discretized systems.
- The system's response to a step input (a sudden change from 0 to a constant value) provides insight into how accurately the discrete system tracks changes. Metrics such as overshoot, settling time, and steady-state error are observed to determine the impact of discretization.

**PROCEDURE:**

1. Click on MATLAB icon and Select Command Window.
2. Enter the program code in the command window.
3. Execute the command *tf* and *c2d* with different frequencies.
4. Obtain the step response of discrete process.

**INFERENCE:**

**RESULT:**

**Problem:**

A system has the following state and output equation.

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ -2 & -3 & 0 \\ 0 & 2 & -3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix} u \text{ and } y = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Check the controllability and observability.

**Matlab Commands:**

>>

**EX.NO:** **TEST OF CONTROLLABILITY AND OBSERVABILITY IN CONTINUOUS AND DISCRETE DOMAIN IN SIMULATION PLATFORM**

**DATE:**

**AIM:**

To determine the controllability and observability of a given system in continuous and discrete domains.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation

**THEORY:**

The continuous-time system is typically represented in the form:

$$\dot{x} = Ax + Bu$$
$$y = Cx + Du$$

Where,

x(t) is the state vector

u(t) is the input vector.

y(t) is the output vector.

A, B, C, and Dare system matrices defining the dynamics of the system.

**CONTROLLABILITY:**

A system is controllable if, through its input signals, it is possible to drive the system from any initial state to any desired final state within a finite period. Mathematically, for a continuous-time linear system described by $\dot{x} = Ax + Bu$, the system is controllable if the controllability matrix Qc =[B AB $A^2B \ldots A^{n-1}B$] is full rank (i.e., its rank is equal to the number of states n). For discrete-time systems, controllability is similarly defined but uses the discrete system matrices.

**OBSERVABILITY:**

A system is observable if, through its output signals, it is possible to infer the entire system's state over a finite time period. For the system $\dot{x} = Ax + Bu$, $y=Cx$ observability is determined by the

**Simulation Response:**

observability matrix Qo = $[C^T \ A^TC^T \ (A^T)^2C^T \cdots\cdots (A^T)^{n-1}C^T]$ , *O* is full rank, the system is observable. Observability for discrete systems follows the same principle, using discrete matrices.

**PROCEDURE:**

1. Obtain the state space model matrix of the system.
2. Determine Qc using formula

$$Qc = [B \ AB \ A^2 B \ldots\ldots A^{n-1} B]$$

3. Check for |Qc| is not equal to zero.
4. Determine Qo using formula

$$Qo = [C^T \ A^TC^T \ (A^T)^2C^T \cdots\cdots (A^T)^{n-1}C^T]$$

5. Check for |Qo| is not equal to zero.
6. Find Rank for both Qc and Qo.

**INFERENCE:**

**RESULT:**

**PROBLEM:**

As an example, consider a third order process. The transfer function of the process is:

$$G(s) = \frac{6.2}{(s+1)(s+2)(s+3)}$$

By using state feedback control $u = -Kx$, it is desired to have the closed –loop poles at *-6, -3+ 4i, -3-4i.* Design a state feedback gain matrix *K* and obtain the closed loop performance with MATLAB.

**MATLAB COMMAND:**

>>

**EX.NO: STATE FEEDBACK AND STATE OBSERVER DESIGN AND EVALUATION OF CLOSED LOOP PERFORMANCE**

**DATE:**

**AIM:**

To design a State feedback and state observer for stabilize the system and meet performance requirements by feeding back the state variables.

**SIMULATION NEEDS:**

1. Software : MATLAB Package
2. Tool : Simulation

**THEORY:**

A state feedback controller is a type of controller that uses the full state vector of a system to compute the control input. In a linear time-invariant (LTI) system, we often represent the system using a state-space model:

$$\dot{x} = Ax + Bu \quad (1)$$

where:

- $x$ is the state vector (representing internal states of the system),
- $u$ is the control input,
- $A$ is the system matrix that describes the dynamics of the states,
- $B$ is the input matrix that describes how the control input affects the states.

In practice, not all states $x$ of a system is measurable. This is a common challenge in control design because effective feedback often requires full knowledge of the system states. A state observer (or state estimator) is designed to estimate the unmeasured states based on the system output and the known input.

Consider a system where the output y is given by:

$$y = Cx \quad (2)$$

Where, $C$ is the output matrix that maps the states to the measured output. A state observer reconstructs the state vector $x$ by using the system's measurements $y$ and the input $u$.

**DESIGN:**

Consider a linear time-invariant (LTI) system in state-space model:

$$\dot{x} = Ax + Bu \tag{3}$$

Using state feedback control law, the control input u is defined as:

$$u = -Kx \tag{4}$$

Where, *K* is the feedback gain matrix.

The closed-loop dynamics become:

$$\dot{x} = (A - BK)x \tag{5}$$

The block diagram of state feedback controller with control gain K is shown in Figure.

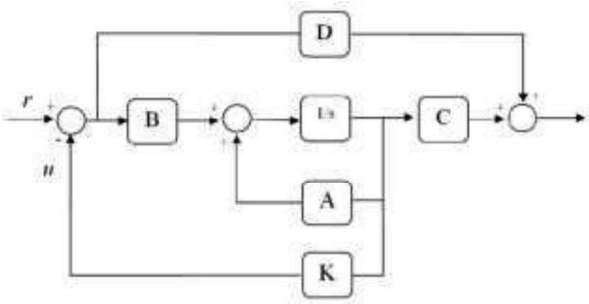



Fig. 1 Block diagram of state feedback control

The eigenvalues of *(A−BK)* determine the closed-loop stability and response characteristics. By choosing *K* properly and achieve the desired stability and dynamic performance of system.

The design of the state feedback gain K is executed in three phases.

i) Choose the desired location of the poles of the closed loop system.

ii) Determine the state feedback vector ensuring the given poles using the Ackermann formula. The Matlab command ***acker*** can be applied.

iii) Determine the compensation factor to fulfil the static requirements.

**PROCEDURE:**

1. Obtain the state space model matrix of the system.
2. Define the desired poles of the closed loop system.
3. Determine the state feedback vector using the Ackermann formula.
4. Find the state space model of a desired closed loop system.
5. Determine the steady state compensation factor.
6. Obtain the open and closed-loop responses of system.

**Simulation Output:**

**INFERENCE:**

**RESULT:**